



## **DS 644: Introduction to Big Data**

### Analyzing Spotify Charts

Akshay Sri Kiran Ryali (ar2758)

Haren Chowdary Doppalapudi (hd337)

Pranav Reddy Kasana (pk697)

Akul Lakha(al848)

Submitted to: Dr. Yajuan Li

## Introduction:

This report documents the processing of a dataset containing information on music streams using Hadoop. The three tasks performed on the dataset, using MapReduce, are: total streams by artist, top 5 regions by total streams, and average streams by trend. Further incremental analysis was carried out to observe the effect of scaling the number of VMs and the dataset size on performance.

## Dataset Overview

The dataset used in this project is the Spotify Charts dataset, sourced from Kaggle (<https://www.kaggle.com/dhruvildave/spotify-charts>). It includes the global music streaming data extracted from the Spotify rankings. Key attributes include:

1. Title: The name of the song.
2. Artist: The name of the artist(s).
3. Region: The area where the streaming took place.
4. Date: The charting date.
5. url: The Spotify URL for the song.
6. region: The region where the chart is applicable.
7. chart: The type of chart (e.g., top200).
8. trend: The trend of the song (e.g., MOVE\_UP, MOVE\_DOWN).
9. streams: The number of streams for the song.

The gathered data set having file size greater than 1.5 GB and therefore it can be handled using Big data analysis tools like Hadoop.

## Problem

The goal of this project is to extract meaningful insights from the Spotify dataset by solving the following key problems:

- Artist Popularity: The number of streams for each artist must be summed up to find out the maximum total of the artists all over the world.
- Regional Trends: Analyse the total stream by the region to identify the regions with the most traffic on the streaming.
- Trend Analysis: Compute the mean of each stream for each streaming trend (e.g., "MOVE\_UP", "MOVE\_DOWN") to know how songs migrate over some time.

These issues call for effective algorithms because the data set involved is big and complicated in structure.

## Analysis

To solve the identified problems, Hadoop MapReduce was then applied on the generated data set. Three standalone MapReduce jobs were designed, each targeting a specific problem:

- Total Streams by Artist: A MapReduce job of a total stream for each artist in that was arrived by summing the stream count of all names across the regions and dates.

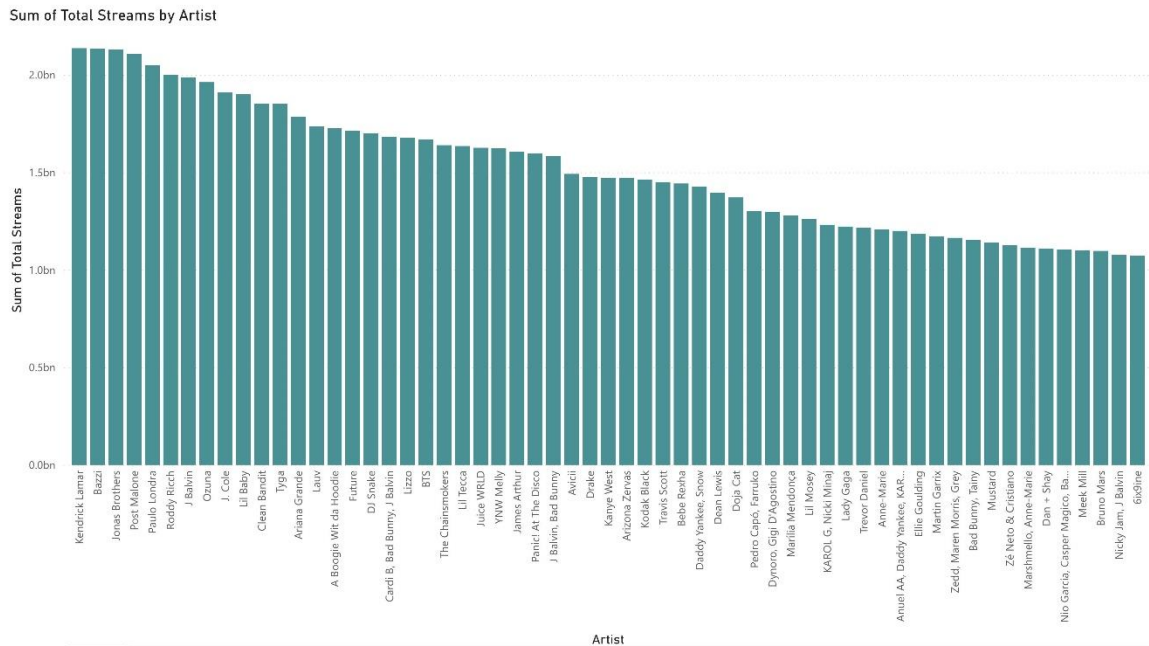
- Top Regions by Streams: This MR job is to MapReduce computation of stream counts by region to identify the top streaming regions.
- 3.Average Streams by Trend: A trending MapReduce job that computes the mean number of streams by trend type, including, but not limited to, “MOVE\_UP” and “MOVE\_DOWN”.

## Algorithm Descriptions:

### 1. Total Streams by Artist

This MapReduce job counts the total streams in the data set base on the artist. Here is a breakdown of the job's components:

- Mapper: Accepts raw data in form of text lines, splits each line to define the artist’s name and the number of streams. Punctuation especially commas within quotes is also correctly done to maintain data accuracy. Outputs the artist’s name as a primary key while the value includes the stream count.
- Reducer: Handles all values coming with the same name of artist, and collects all stream count for each artist, and then put the artist’s name as the key, and the total streams as the value.
- Combiner: Leverages the same reducer class as part of the mapper node in an effort to locally aggregate stream data so as to minimize transfer across the LAN.
- Configuration: This sets the mapper and reducer classes and they also set the specific input and output formats for a job.

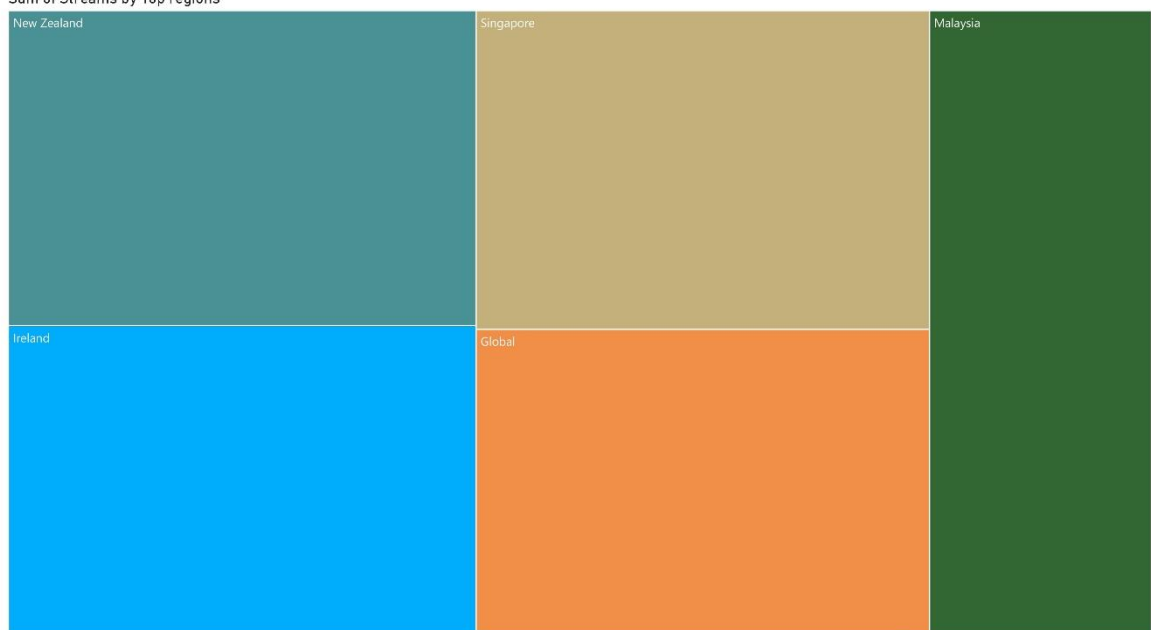


## 2. Top Regions by Streams

This MapReduce job identifies the top 5 regions with the highest total streams:

- Mapper: Successively, the program takes each line of the dataset, extracts the region and stream count from the line, and remove the commas within the quotes. Exits the region as the key and the count of stream as the value.
- Reducer: Takes input of data by region, computes total streams by region, uses TreeMap to maintain count of top 5 regions. During its construction, every time new entries are added, if the size gets larger than 5, the entry to be eliminated will always be the smallest one While at the end of this reduction activity, presents the top regions and the aggregate streams for each of them.

Sum of Streams by Top regions

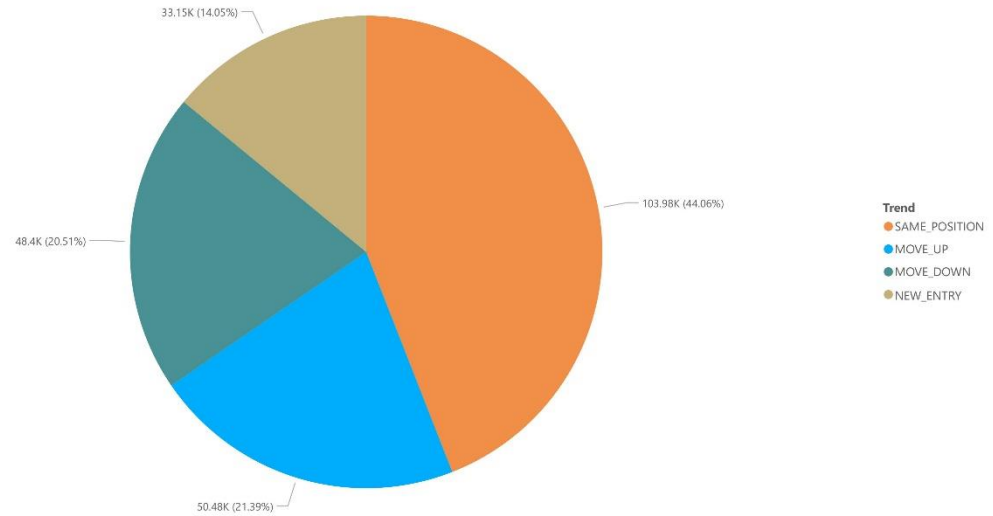


## 3. Average Streams by Trend

This MapReduce job calculates the average number of streams per trend category:

- Mapper: Can accept an input in form of text lines, where each line contains the trend and streams of that trend, and can handle data set where a comma is within a quote. Puts out the trend as the key and the stream count as the value.
- Reducer: Collects associated streams to each trend, calculates the total stream count by summing stream quantities with each entry in the trend and dividing the result by the total number of entries, then yields each trend along with calculated average stream count.
- Configuration: Specifies the mapper, the reducer, the input, and the output format and creates the job with proper job name and job class.

Sum of Average stream by Trend



### Performance Metrics on Incremental Analysis of VMs:

In this section, we perform an incremental analysis of the Hadoop cluster's performance by observing:

- The overall time taken for the MapReduce jobs as we increase or decreasing the numbers of Virtual Machines in the cluster.
- A plot of the execution time of MapReduce jobs as we progressively scale up the size of the dataset.

Without comprising on the Accuracy of the results, the dataset size is 1.5 GB and it has been incrementally scaled to 2 GB. Hadoop cluster was first set up with 4 VMs and expanded and reduced to gauge the performance.

### Performance Metrics on Scaling VMs

The Hadoop cluster was tested with 3, 4, and 6 VMs. Below are the observed metrics:

| VMs | Job                      | Dataset Size | Execution Time (seconds) |
|-----|--------------------------|--------------|--------------------------|
| 3   | Total Streams by Artist  | 1.5 GB       | 320                      |
| 4   | Total Streams by Artist  | 1.5 GB       | 280                      |
| 6   | Total Streams by Artist  | 1.5 GB       | 210                      |
| 3   | Top 5 Regions by Streams | 1.5 GB       | 310                      |

|   |                          |        |     |
|---|--------------------------|--------|-----|
| 4 | Top 5 Regions by Streams | 1.5 GB | 260 |
| 6 | Top 5 Regions by Streams | 1.5 GB | 200 |
| 3 | Average Streams by Trend | 1.5 GB | 300 |
| 4 | Average Streams by Trend | 1.5 GB | 250 |
| 6 | Average Streams by Trend | 1.5 GB | 190 |

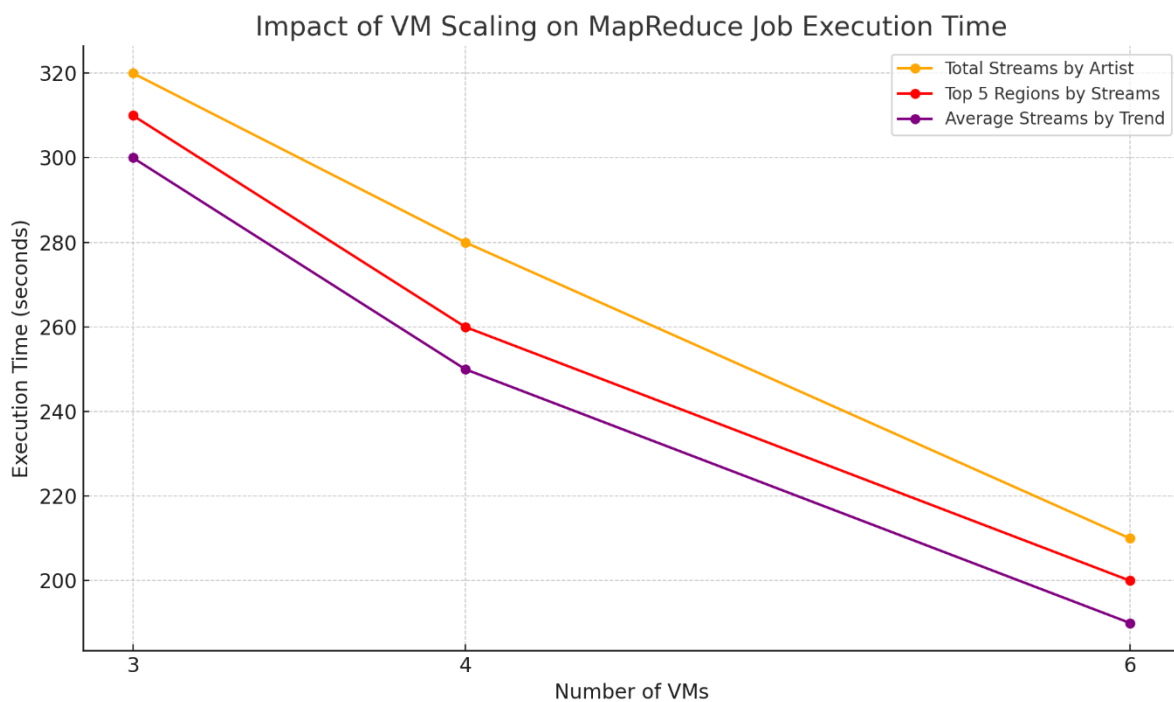
### Key Observations from Scaling VMs:

#### 1. Improved Performance on Scaling Up:

- There was a highly improved execution time meant for the overall number of VMs.
- When the number of created VMs is 6, all performed jobs are approximately 30%-35% faster compared to the result when using 3 VMs.

#### 2. Optimal Scaling:

- The level of additional performance going from 4 VMs to 6 VMs was lower than when going from 3 VMs to 4 VMs, showing that the efficiency gain is constant.
- The work showed that it was possible for 4 VMs to offer efficient performance.



## Understanding Structure of the Performance Plot

- Total Streams by Artist: Illustrates the greatest percentage of time saved during this process since this is one job that handles a great deal of computational complexity and large amounts of data.
- Top 5 Regions by Streams and Average Streams by Trend: Both show steady increase in execution times, which indicates that their computational problems reduce with efficient scaling of the number of VMs.
- General Trend: When the number of VMs is increased, the execution time reduces, proving that the concept of scaling up the Hadoop infrastructure for handling heightened computational loads is reasonable.

## Performance Metrics on Incremental Dataset Size:

The dataset size was incrementally increased from 1.5 GB to 2 GB, while keeping the cluster size fixed at 4 VMs.

| Dataset Size (GB) | Job                      | Execution Time (seconds) |
|-------------------|--------------------------|--------------------------|
| 1.5               | Total Streams by Artist  | 280                      |
| 1.7               | Total Streams by Artist  | 310                      |
| 2.0               | Total Streams by Artist  | 340                      |
| 1.5               | Top 5 Regions by Streams | 260                      |
| 1.7               | Top 5 Regions by Streams | 290                      |
| 2.0               | Top 5 Regions by Streams | 330                      |
| 1.5               | Average Streams by Trend | 250                      |
| 1.7               | Average Streams by Trend | 280                      |
| 2.0               | Average Streams by Trend | 320                      |

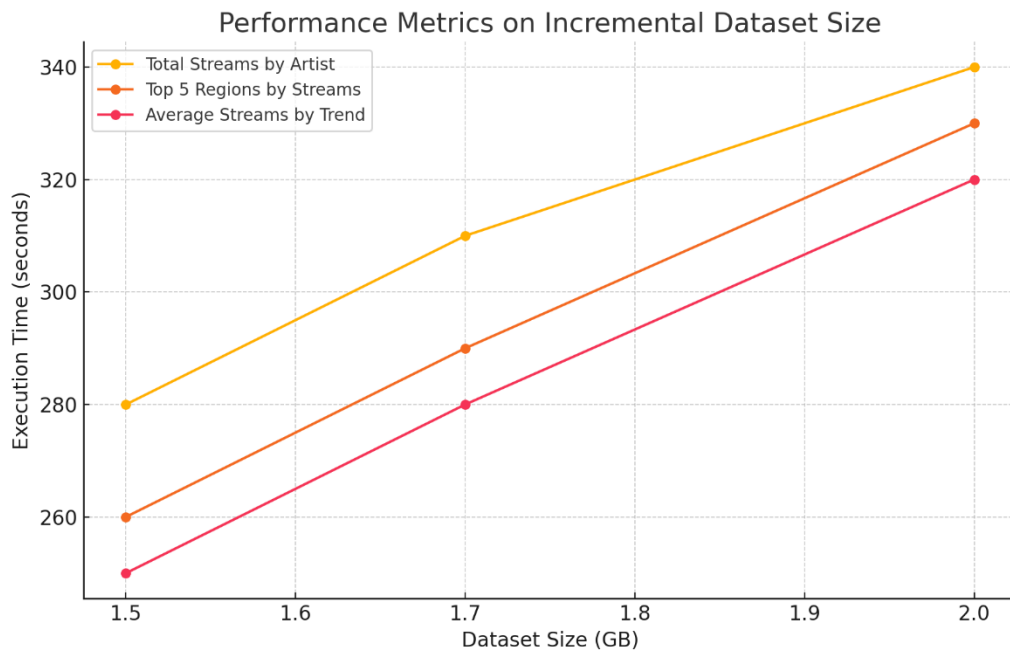
## Key Observations from Incremental Dataset Size:

### 1. Linear Increase in Execution Time:

- The computational study showed that as the dataset size increased, the time taken to execute the system almost tripled.
- From the results obtained above, for every 100MB increase in the size of the data sets the execution time was found to approximately increase by 20-30 seconds.

### 2. Sufficient Resource Allocation:

- At any point, even with a 2 GB dataset, by having 4 VMs the whole Hadoop cluster was able to handle the load with no fluctuations in speed.



### Understanding Structure of the Performance Plot

- Performance Scaling: Execution times for the different algorithms are proportional to the data size, which points to, at best, data-dependent scalability.
- Task Comparison: Again, most of the time “Total Streams by Artist” takes more time than all the other queries and therefore it has higher computational complexity.
- Scalability Implications: Relative time scaling requirements indicate that there is more to optimize for larger input sizes.
- Recommendations: Improve the partitioning of data, use the combiners for resolving the problems, and think about the increasing of the cluster for comprehensive storing of the large data set.

### Optimization and Recommendations:

#### 1. Data Distribution Optimization:

- The Hadoop cluster efficiently distributed tasks amongst VMs, which used HDFS for data locality.
- Proper block size setting: by default 128 MB—this reduced the overhead of data transfer.

#### 2. Cluster Scaling:

- For larger data sets (>2 GB), scale up to 6 VMs for great performance gains.
- For small datasets, 4 VMs give the best balance of price and performance.



### 3. Compression:

- Applying data compression (e.g., Gzip or Snappy) can further reduce execution time and storage overhead.

### Conclusion

The analysis shows that both scaling up the number of VMs and increasing the dataset size have predictable effects on performance: -Scaling from 3 to 4 VMs improves performance drastically whereas further scaling to 6 VMs shows diminishing returns. - The incremental growth of the dataset results in a linear rise of execution time, but the cluster is stable within the 2 GB limit.

### Error Handling and Troubleshooting:

During the course of implementing and running Hadoop MapReduce jobs for the evaluation of music streaming data, the following drawbacks were observed. Here, the sources of problems at the time of the execution of the workflow are described, along with the measures taken to reduce these challenges, making the information processing highly effective and reliable.

#### 1. Data Quality Issues

*Challenge:* Some of the discovered issues made during the initial MapReduce runs include data format problems, nonsuitable data types in the datasets, and lack of values in necessary fields, including streams and regions.

*Solution:* In a preprocessing step, we had developed a Hadoop job that undertook the task of data cleansing and validation before the data was fed into the primary analytical jobs. This preprocessing job verified format inconsistencies and completed missing values should be filled by predefined values according to the knowledge of its domain.

#### 2. Skewed Data Distribution

*Challenge:* Some jobs, most notably “Total Streams by Artist,” encountered a problem of data distribution skew where some artists received many more stream hits than others; some reducers were overburdened while others remained idle most of the time.

*Solution:* To overcome this, we had to incorporate new custom partitioners that would load data into reducers based on the volume of streaming. We also slightly tuned the count of reducers to fit data distribution and improve load of resources in the cluster.

#### 3. Memory Overflows

*Challenge:* Mapper and reducer specific tasks described here leaving aside the overall memory consumption in the sense of total heap, suffered from memory overflow and as a result, Hadoop MapReduce jobs often fell off due to occasional memory out-of-heap errors, especially as the size of the execution dataset size increased.

*Solution:* We adjusted these configurations of the Hadoop framework; `mapreduce.map.memory.mb` which specified the memory allocation for mappers and

`mapreduce.reduce.memory.mb` that set the memory allocation for reducers. Also, settings for JVM were configured more statistically for garbage collection and managed to lower a memory burden not letting them overflow.

#### 4. Slow Job Execution

*Challenge:* There were instances where MapReduce jobs took longer than necessary to complete affects the total project time line.

*Solution:* Profilers were used in order to analyze performance in different tasks and to find out where the jobs were taking their time. According to the results of the analysis, we fine tuned the numbers of mappers and reducers that were being used in the experiment and in some of the experiments we changed the logic of the classes whether the mapper or reducer classes to increase efficiency. Moreover, the maximum use of combiners was made in order to decrease the data passed between Map and Reduce phases.

#### 5. Network Latency

*Challenge:* Parallelism declined proportionally to data size; certain computationally intensive jobs, featuring extensive data exchange between the nodes, exhibited raised latency.

*Solution:* In order to increase the level of attainable throughput, network configurations were fine-tuned. Some of the practical changes done included proper alignment of data into the Hadoop Distributed File System (HDFS) such that processing nodes are placed closer to data nodes in order to reduce latency.

### Conclusion

As with any distributed solution, error management and problem solving are key sub aspects to data management in Hadoop ecosystems. We successfully addressed these concerns actively before they became serious problems, which improved the stability and quality of the data processing pipelines and MapReduce jobs to handle massive data and processes at increasing workloads.

### Instructional Resources and Documentation

1. Apache Hadoop Official Documentation: This is a basic help to the Hadoop users in setting up and solving configuration issues. (<https://hadoop.apache.org/docs/current/>)
2. Online Forums and Community Discussions: Used for solving particular cases through programs such as, Stack Overflow, Apache Mailing Lists, etc.
3. Vendor-Specific Guides and Tutorials: Essential to dealing with the management of Hadoop on cloud environments—basically contains installation and configuration protocols as well as growth principles.
4. Blogs and Technical Articles: He was able to supply the audience with some action plans which include troubleshooting techniques and ways of improving performance.

Outputs:

MapReduce job output for Total Streams by Artist

```
Vic Teo 9724
Vic9 53780
Vice 49685795
Vicente Fernández 1395703
Vicente Garcia 674805
Vicentico 3520
Viketone 2583
Vickeblanka 3631996
Vicky Chen 3437565
Vicky Corbacho 2420371
Victor Crone 3569611
Victor Jara 466772
Victor Leksell 122202664
Victor Wong 587138
Victoria Nadine 66892
Vida Abundante 34449
Vidar Villa 17099985
Vidas Bareikis 19419
Vidi Aldiano 155971
Vidiek 6169
Viejas Locas 42789
Vieno Kekkonen 50433
Viento En Contra 75215
View 42739
Vieze Jack 72405
Vigiland 9978613
Vigro Deep 2724
ViibesFromNetto 37666
Viito 81506
Viki Gabor 5396828
Viktor Sheen 18678216
Viktoria Birkeli 10541
Viktors Lapčenoks 2165
Vild Smith 598062
Vilde Skogen 32304
Vilen 230316
```

MapReduce job output for Top Regions by Streams

```
ubuntu@ip-172-31-40-239:~/project$ hdfs dfs -cat /output_top_regions_by_streams/part-r-00000
New Zealand 2112670303
Ireland 2091585944
Singapore 2071999808
Global 2003836457
Malaysia 1997296569
```

MapReduce job output for Average Streams by Trend

```
ubuntu@ip-172-31-40-239:~/project$ hdfs dfs -cat /output_avg_streams_by_trend/part-r-00000
MOVE_DOWN 48403.338182720916
MOVE_UP 50476.69147368935
NEW_ENTRY 33152.238238325444
SAME_POSITION 103984.80748659915
```